Reinforcement Learning Method to Optimise Traffic Signal Control in Typical Urban Four-Way Intersection Using SUMO Traffic Simulator

Alex Wang (z5420393), Dhwanish Kshatryia (z5421168), Rohan Agarwal (z5423480), Evan Xiong (z5421149), Michael Wang (z5421148)

Abstract — **Inefficient and inflexible traffic signal control at urban intersections contributes to congestion, delays, fuel wastage, and increased emissions. The consequences of poor traffic control have become increasingly urgent in Sydney due to rapid urban population growth. Optimizing traffic signals remains a significant challenge in urban traffic management. Recent advances in reinforcement learning (RL) offer promising solutions to address this problem. In this paper, we present RL-based approaches for traffic light signal optimization at a typical four-way urban intersection, implemented using the Simulation of Urban Mobility (SUMO) platform. We employ Deep Q-Networks (DQN) to learn optimal signal policies from real-time traffic data. Two model variants are explored: a lane-detector-based DQN and an image-based DQN using a convolutional neural network (CNN) with a bird's-eye view of the intersection. The lane-detector-based model uses queue lengths state representations, while the image-based model takes the last four vehicle coordinates as its states. Simulation results show that our approach converges reliably and achieves significant reductions in average queue length and wait time compared to benchmark methods, however, struggles with maximum wait time situations which the current real world fixed cycle traffic light operation overcomes. Comparing the lane-detector-based DQN against the image-based DQN indicates that the fully connected layer performs better. Overall, these models demonstrate strong potential for integration into intelligent urban traffic management systems, although further investigation and analysis are required to optimize traffic flow and resolve ethical dilemmas.**

Keywords—Reinforcement Learning, Traffic Light Signal Control, Reward, Wait Time, Queue Length, SUMO, Experience Replay. Target Networks, Deep Q-Networks

I. Introduction

Traffic light control is a critical component of urban traffic management. It is responsible for regulating the movement of vehicles through intersections. As urban populations grow and vehicle usage increases, poorly optimised traffic signals contribute significantly to congestion, fuel inefficiency, and elevated greenhouse gas emissions. The importance of adaptive and intelligent traffic control is therefore more pressing than ever before as with it we can achieve environmental sustainability, higher levels of economic productivity and commuter well-being.

Traditionally, traffic systems are often based on fixed intervals or simple reactive mechanisms. While these approaches are relatively easier to implement, they lack the flexibility to respond to dynamic and often unpredictable real-world traffic conditions. As a result, intersections controlled by conventional systems frequently experience inefficiencies, especially during peak hours.

The aim of this paper is to consider adopting a reinforcement learning approach, specifically a Deep Q-Network (DQN), to optimise traffic light control at a typical four-way intersection. This approach learns the optimal signal policies by interacting with its environment, observing the state of traffic, and receiving feedback in the form of rewards. Unlike the previous approaches, this model can learn about diverse traffic patterns.

Our contribution lies in developing a DQN-based traffic control system trained and evaluated using the SUMO simulator. To simulate a dynamic environment, the model is trained on three urban traffic scenarios P1-P3, (defined in the method III), then evaluated on traffic scenario P4.

By leveraging reinforcement learning, this project demonstrates a promising step towards more adaptive, data-driven approaches to traffic light control.

II. Related Work

In the past two decades, there have been several attempts to optimize traffic flow. Early models like El-Tantawy et al. [4] were limited to Q-learning and a discrete state space, with queue length of an incoming lane categorized into "low", "medium", and "high" levels. Due to this, the complexity of the traffic environment at a traffic intersection could not be fully represented, limiting adaptability in diverse and unpredictable traffic conditions.

Guo Wang et al. [3] addresses the challenge of inefficient and inflexible urban traffic control by proposing a reinforcement learning approach using neural networks as Q-function approximators. States are defined from real-time traffic data, and actions represent different traffic signal phases. This approach was tested in SUMO with various traffic patterns. Currently, the models used outperform the benchmarks. However, there is room for further improvement for the Q-network by tuning its performance in terms of convergence and stability, as well as comparison with other methods. Drawing from this paper, we extended Guo Wang's approach, comparing the model against a convolutional neural network (CNN) DQN.

Minh et al. [2] combines reinforcement learning with convolutional neural networks to learn directly from raw pixels, achieving human performance on Atari 2600 games. Key innovations included experience replay, target networks, and CNN architecture processing stacked game frames. However, unlike these closed-world experiments, we incorporate real-world constraints (e.g., traffic rules, yellow light transition lengths, or fairness to rarely used lanes) and train on multiple different traffic patterns (e.g., tidal peak hour, major/minor road, etc.) when building our DQN.

Regarding the environment used for training, we drew inspiration from Lopez et al. [1]. Their paper, Microscopic Traffic Simulation Using SUMO outlines SUMO's capabilities for generating, validating, and evaluating large-scale traffic scenarios. Using a real-world case study in

Bologna, it demonstrates SUMO's ability to simulate complex urban traffic. However, preparing realistic microscopic simulations is time-consuming due to data mismatches, and accurately modelling traffic breakdown remains challenging with different models producing varying predictions of congestion pattern and capacity drops. As a result, we used a less complex intersection for our simulations due to limited computational resources, allowing us to focus more on refining the model design.

III. METHODS

In our project, the environment was simulated using SUMO. It consists of a 4-way intersection with two lanes in each direction, with through-lane and turning movements. This setup is depicted in Figure 9. To model vehicle arrivals realistically, we configured our vehicle arrival rates to follow a Poisson distribution. This created a stochastic arrival process, introducing variability in traffic demand and thereby ensuring that all simulation runs are different due to randomized arrival rates.

To train and evaluate our RL agent, we designed four distinct traffic patterns which were proposed in the paper by Guo, M. (2019). These patterns are made up of unique combinations of lane-specific arrival rates and temporal demand profiles (Figure 8). This exposes the agent to dynamic (stationary and moving) traffic conditions. The traffic patterns are as follows.

- **P1** - Major/Minor Road Traffic: One road has significantly higher traffic volume than the other, representing a major road intersecting with a side street.
- **P2** - Through/Left-Turn Lane Traffic: Traffic demand is unevenly distributed between straight-through and left-turning lanes.
- **P3** - Tidal/Rush Hour Traffic: Traffic is directionally biased, with heavy flow in one direction for part of the simulation, then reversing later, simulating rush-hour effects.
- **P4** - Varying Traffic Demand: Traffic demand fluctuates unpredictably during the episode.

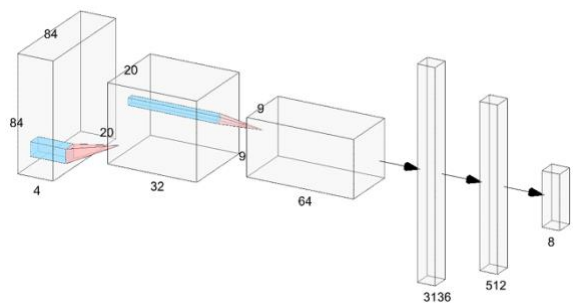
To promote generalization, our RL agent was trained in patterns P1, P2 and P3. Unlike the original approach in Guo (2019), which trained separate agents for each pattern, we trained a single agent for all three patterns. This ensured our agent learned under structured but varied conditions. Pattern 4 in contrast was used as our unseen test set and hence the agent's performance here clearly depicted its generalization ability to dynamic traffic scenarios.

First, a baseline model was created which utilized a fixed traffic control schedule that mimics the behavior of NSW traffic control. This model transitioned between 4 phases in the order specified in Figure 10, with a 6 second green light stage and a 4 second green light stage in each phase. This model served as a reference for comparing our RL models.

The task was then modelled as a Markov Decision process (S, A, R), where the actions were defined as eight non-conflicting traffic phases as defined in **Error! Reference s ource not found.**9. Then, the reward was defined as the negative total queue length at each timestep. Both these definitions were adopted from the investigations by Guo et al [3]Furthermore, we implemented experience replay and target networks in alignment with Mnih et al [3]to stabilize learning and address the correlation and non-stationarity issues faced during RL approximation.

Two potential state definitions were considered. The first state model S_1 was taken as the $\mathbb{R}^8$ vector of per-lane queue lengths, which captures the spatial characteristics of traffic (congestion). A dense DQN adapted from Guo et al [3] was used to approximate Q-values under S_1. Its architecture, shown in Figure 12, consists of two fully connected hidden layers followed by a linear output over each of the eight actions. This model was chosen as a computationally efficient and stable function approximator capable of producing strong results in multi-pattern signal control over lower dimensional features, as shown in Guo et al [3]. This stability is highly desirable in a real-world traffic scenario as overfitting traffic fluctuations can lead to significant undesirable outcomes such as an increase in inefficiency and unsafe driving. However, to model more complex spatial and temporal relationships within the traffic dynamics, a more complex state and approximator would be required.

The second state model S_2 was taken as an 84×84 occupancy grid of vehicle coordinates over the last 4 consecutive seconds (visualized in Figure 15). This extends the first model by capturing both the spatial and temporal dynamics of the traffic, allowing the agent to make decisions based on factors such as congestion, traffic propagation and the formulation of traffic platoons. A convolutional DQN adapted from Mnih et al [2] was used to approximate Q-values under S_2. A CNN architecture is most appropriate for processing these $4 \times 84 \times 84$ inputs due to the convolutional filters which can efficiently extract spatial relationships within the occupancy grip. The architecture consists of three convolutional layers, followed by a fully connected layer and finally a linear output over each of the eight actions (as shown in Figure 13). This model trades stability of the dense network in favor of granularity in traffic control.



S2 Architecture Diagram

IV. SIMULATION EXPERIMENTS

This section presents the simulation experiment which includes (1) the SUMO simulation environment, (2) the metrics used to evaluate the tuning, training, and final performance, (3) and the final tuned hyper-parameters of both models.

(1) Our final experimental setup didn't use hardware acceleration as the model running speed bottleneck was in SUMO. This imposed significant time constraints, prompting us to stick to testing models in our papers of new architecture. We also limited the hyperparameter tuning.

For our code running environment, we used the SUMO and TraCI (Traffic Control Interface) python libraries. Our machine used a 13th Gen Intel Core i7-1360P processor and 16 GBs of RAM on Windows 11 10.0.26100 build 26100. The code was implemented in Python 3.12 using PyTorch version 2.7.1 and SUMO version 1.23.0. Table 1 in the appendix details the specific SUMO environmental details in our simulations.

(2) We chose 3 main evaluation metrics for model interpretability and to help us tune values, analyze training status and evaluate final model performance. These metrics were collected during hyperparameter tuning, model training, and model evaluation.

Average reward acted as a proxy for overall flow/throughput (equivalent to negative queue length). It is measured in number of cars in queue. Average wait time was a direct reflection of driver experience, as well as direct interpretability of model performance. It is measured in seconds. Finally, maximum wait time was a check for worst-case fairness and tail risk, to see if in the process of optimizing the model reward the model creates situations where the minority car is prevented from moving for a very long time. This is also measured in seconds.

(3) Next hyperparameter tuning. We performed targeted grid search focusing on gamma, epsilon decay and memory buffer size was performed over 200 episodes for each model. Below is a table representing the ideal hyperparameters for the dense DQN model. The CNN DQN model table is provided in the appendix. Further analysis and explanation for these values can be found in the project notebook.

The model with the best combination of hyperparameters provided the best average reward (highest), lowest average waiting time and lowest maximum waiting time. Using these values, we trained our models, this brings us to the training results and final model evaluation.

V. RESULTS

The two RL agents were trained on traffic patterns P1 to P3 (see Methods), cycling through patterns each episode. Pattern P4 was reserved as the unseen test set for evaluating generalization. The training was done over 200 episodes, where one episode represents one full simulation, until every vehicle has finished their routes, on average approximately 36 seconds per episode for the CNN DQN training and 18 seconds per episode for the dense DQN model.

Figure 1 and Figure 2 show the episode reward as it changes and fluctuates throughout the training process for the dense DQN model and the CNN model respectively. Notably during training, the CNN DQN was relatively unstable with spikes and jumps in reward. The CNN DQN model also had approximately double the convergence time of the dense DQN model.

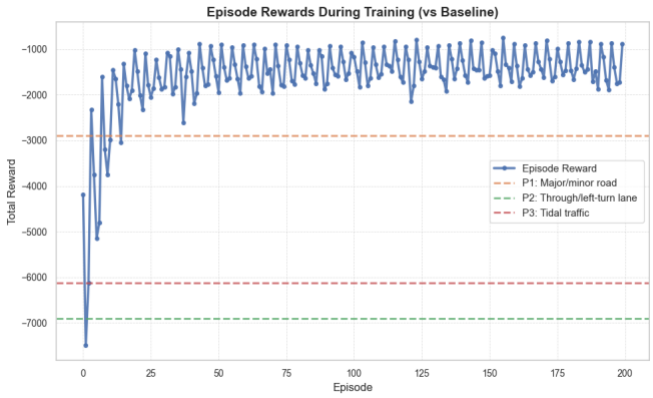


Figure 1: Average Reward Training Dense DQN

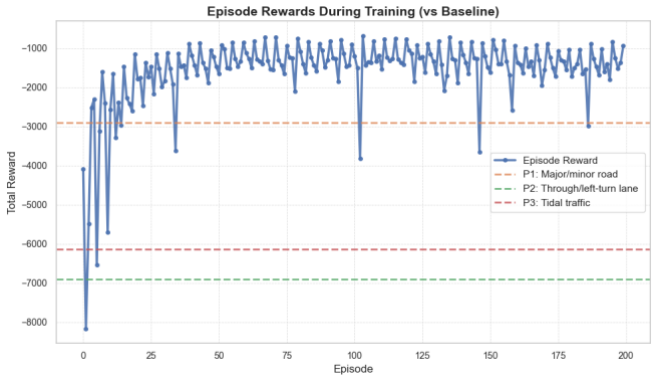


Figure 2: Average Reward Training CNN Model

Figure 3 and Figure 4 below show that RL models perform very poorly for the maximum wait time metric. There is no real convergence, and there are huge fluctuations throughout the training process, with some very extreme values being reached by both models.

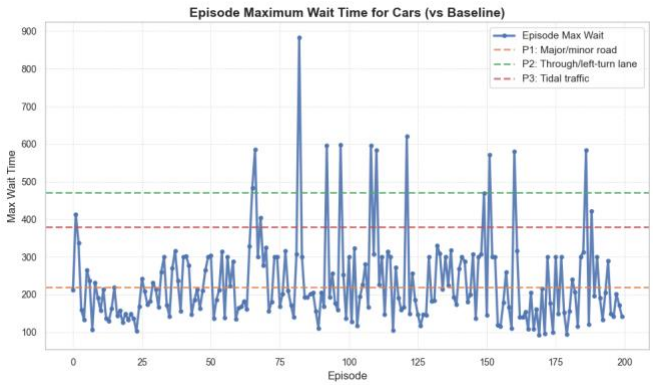


Figure 3 - Dense DQN Max Waiting Time (Training)

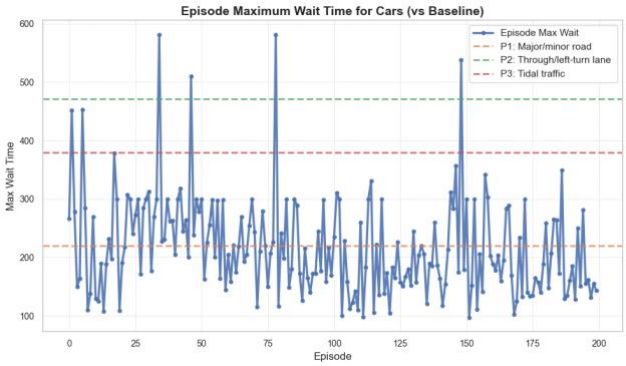


Figure 4 4– CNN DQN maximum waiting time metric during training

We ran both trained models on 100 episodes for each traffic pattern P1 – P4. The performance of the RL models was evaluated against the (non-adaptive) baseline model.

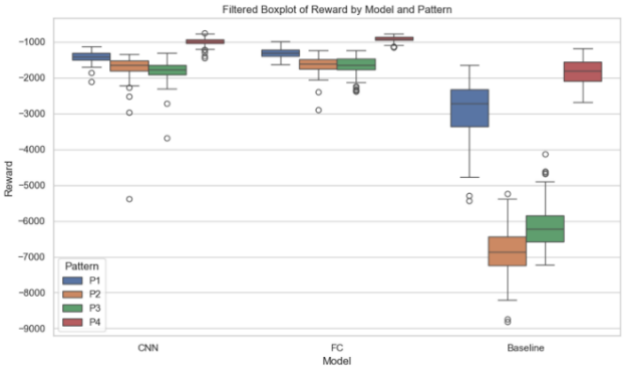


Figure 5 - Model Evaluation Results (Outliers Removed) - Reward

Figure 5 compares each model's performance in optimizing reward. As expected, both reinforcement learning agents significantly outperform the baseline model, with the dense model performing best. The CNN model exhibited higher variance, including several outliers having particularly poor performance, suggesting the CNN struggled to accurately approximate Q-values across the full action–state space, leading to suboptimal decision-making in certain scenarios.

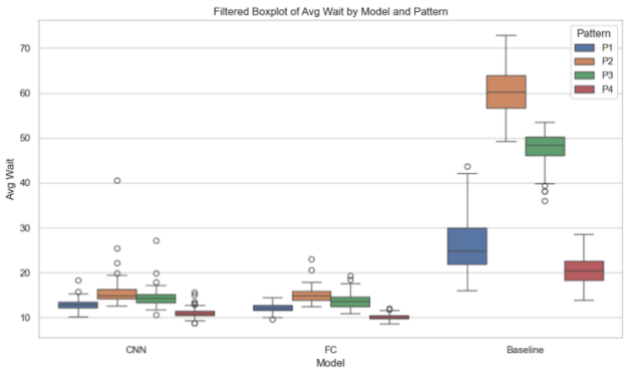


Figure 6 - Model Evaluation (Outliers removed) – Average Wait

While the wait time is not explicitly incorporated into the reward, an intersection with short queues should also theoretically have lower average waiting times. Figure 6 supports this, as it shows both RL agents with lower average waiting times. This aligns with the results of Guo et al. (2019).

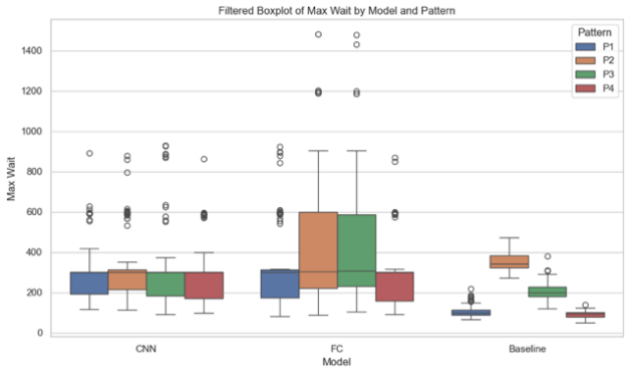


Figure 7 - Model Evaluation (Outliers removed) – Maximum Wait

In contrast, Figure 7 shows that our RL agents perform poorly relative to the baseline on maximum waiting time. To maximize the reward, the agent likely chooses traffic lights that allow the busiest lanes to pass first. In scenarios where there are less congested lanes with only a few vehicles, these cars may be left waiting indefinitely (see Figure 16), since prioritizing them does little to improve the total reward. These findings suggest that, despite their effectiveness at reducing queue lengths and average waiting times, RL agents may not be suitable for deployment in real-world traffic systems due to the sacrifice of maximum waiting times.

VI. CONCLUSION

Our paper examines a metric that past papers such as Guo et al. (2019) did not consider — specifically the maximum waiting time of vehicles in the intersection. Our results demonstrate that a policy optimized solely to reduce total congestion may inadvertently increase waiting times for vehicles in low-traffic lanes, revealing a trade-off between overall efficiency and equitable traffic flow management. This presents an ethical dilemma — should we prioritize the collective benefit of minimizing traffic congestion or ensure fairness for everyone. Future studies can look at different reward functions that incorporate maximum waiting times as a metric.

Furthermore, our paper also proposes a separate model which uses a CNN, following the approach of Mnih et al. (2015) where raw visual inputs were used for deep reinforcement learning. Theoretically, its state representation should be able to capture spatial information and vehicle velocities and exceed the performance of the dense DQN model. However, it appears unstable with occasional extreme outlier poor performances and underperforms slightly on average when compared to the dense DQN model, possibly due to a lack of architecture and hyperparameter tuning from computing constraints. Future studies could explore a wider variety of CNN architectures and experiment with more informative visual representations of the state – for example including not only vehicle positions but also traffic light locations, signal phases, and intersection layouts. Our current model only encodes vehicle coordinates onto a black grid, which may limit CNN's ability to learn nuanced traffic dynamics.

Appendix

Traffic pattern	Direction	Arrival rate[a] (vehicles per second)		
		Through	*Left -turn*	*Right-turn*
P1: Major/minor road	N-S	0.05	0.025	0.01
	S-N	0.05	0.025	0.01
	E-W	0.1	0.05	0.01
	W-E	0.1	0.05	0.01
P2: Through/ left-turn lane	N-S	0.05	0.025	0.01
	S-N	0.05	0.025	0.01
	E-W	0.05	0.1	0.01
	W-E	0.05	0.1	0.01
P3: Tidal traffic	N-S	0.1	0.08	0.01
	S-N	0.05	0.025	0.01
	E-W	0.1	0.08	0.01
	W-E	0.05	0.025	0.01
P4: Varying demand traffic	N-S	0.05	0.025	0.01
	S-N	0.05	0.025	0.01
	E-W	0.05 (0-1200 steps) 0.15 (1200-1800 steps)	0.025	0.01
	W-E	0.15 (0-600 steps) 0.05 (600-1800 steps)	0.025	0.01

Figure 8- Traffic Pattern Design (from Guo et al. (2019)

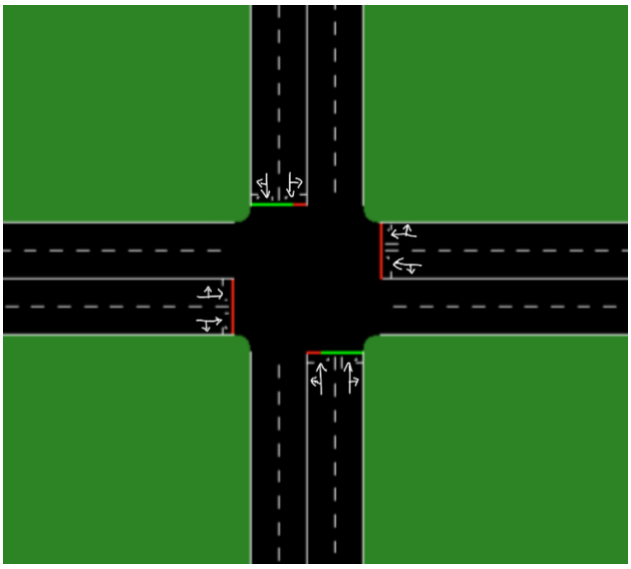


Figure 9- Intersection design

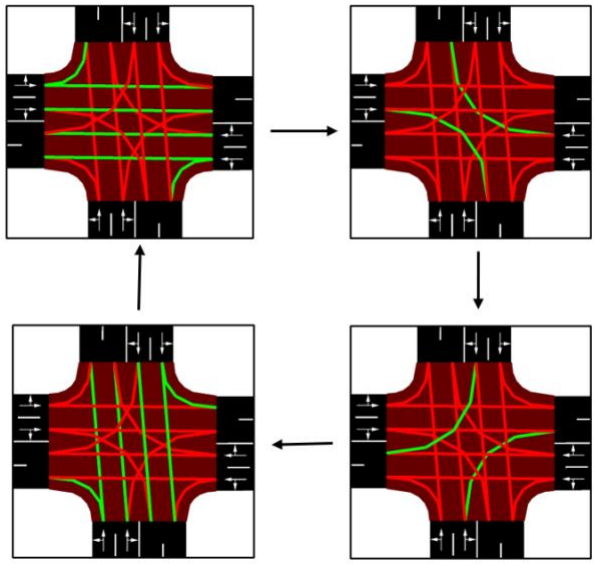


Figure 10 - Baseline Model Phases

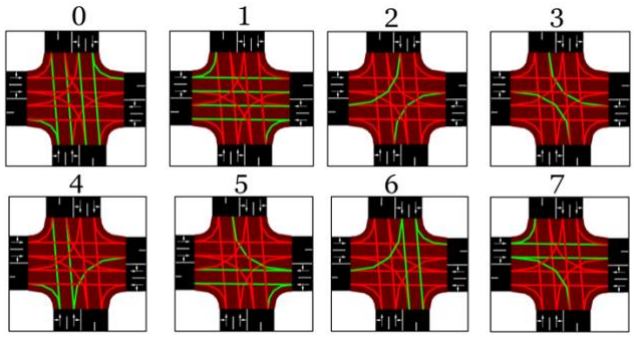


Figure 11- RL Action Space

Layer	Dimensions	Details
Input	8	Take s_t as an input
Hidden 1	128	ReLU Activation
Hidden 2	64	ReLU Activation
Output	8	Linear output over 8 potential actions

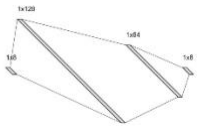


Figure 12 - Dense DQN Architecture

Layer	Shape	Details
Input	(4, 84, 84)	Take s_t as an input
Conv 1	(32, 20, 20)	8×8 kernel with $s = 4$ & ReLU Activation
Conv 2	(64, 9, 9)	4×4 kernel with $s = 2$ & ReLU Activation
Conv 3	(64, 7, 7)	3×3 kernel with $s = 1$ & ReLU Activation
Flattening	3136	N/A
FC	512	ReLU Activation
Output	8	Linear output over 8 potential actions

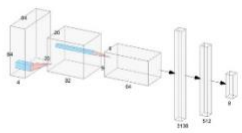


Figure 13- Conv DQN Architecture

Figure 14 - Conv DQN Architecture Diagram

Figure 15 - S2 State Visualization

Parameter	Value

Lane length	150 meters
Vehicle length	5 meters
Min gap between vehicles	2.5 meters
Car-following model	Krauss following model [22]
Max vehicle speed	13.42 m/s
Acceleration ability of vehicles	2.6 m/s2
Deceleration ability of vehicles	4.5 m/s2
Duration of yellow signal	4 seconds
Time span Δ signal phase	6 seconds

Table 1: SUMO Environment Details

Hyperparameter	Search Range	Final Value
γ	0.99, 0.995, 0.999	0.99
ε	0.9 (fixed)	0.9
ε decay	0.95, 0.97, 0.99	0.95
Minimum ε	0.05 (fixed)	0.05
Learning Rate	0.01 (fixed)	0.01
Batch Size	128 (fixed)	128
Target Update Frequency	1800 (fixed)	1800
Memory Size	10,000, 20,000	20,000

Table 2: Hyperparameters for the Dense DQN

Hyperparameter	Search Range	Final Value
γ	0.99, 0.995, 0.999	0.999
ε	0.9 (fixed)	0.9
ε decay	0.95, 0.97, 0.99	0.95
Minimum ε	0.05 (fixed)	0.05
Learning Rate	0.01 (fixed)	0.01
Batch Size	128 (fixed)	128
Target Update Frequency	1800 (fixed)	1800
Memory Size	10,000, 20,000	20,000

Table 3: Hyperparameters for the CNN DQN

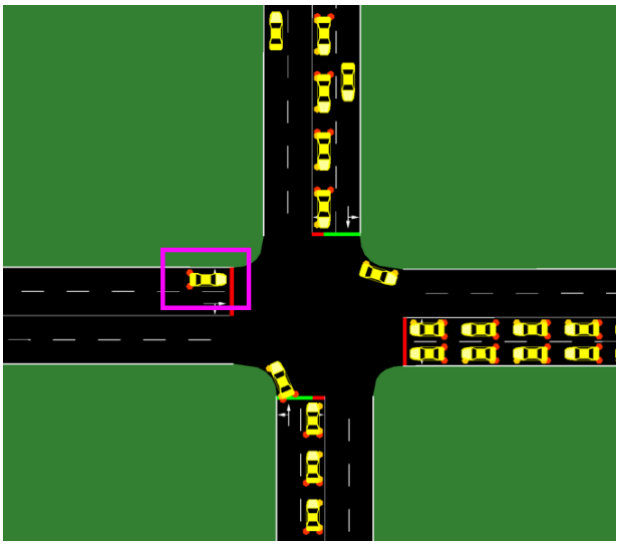


Figure 16 - Scenario where car in the less congested lane is left waiting indefinitely

Model	Pattern	Episodes	Avg Reward	Avg Wait	Max Wait
FC	P1	100	-1311.67	12.10	922.00
	P2	100	-1641.54	15.09	1481.35
	P3	100	-1655.37	13.76	1478.40
	P4	100	**-919.46**	**10.18**	869.15
CNN	P1	100	-1411.14	12.95	891.45
	P2	100	-1734.91	15.72	877.70
	P3	100	-2994.02	20.07	**1773.75**
	P4	100	-997.74	11.03	861.50
Baseline	P1	100	-2901.72	26.42	219.55
	P2	100	**-6904.53**	**60.70**	471.20
	P3	100	-6131.47	47.68	379.95
	P4	100	-1838.72	20.61	**139.20**

Figure 17 - Average results from model evaluation (100 simulations on each pattern)

References

[1] Lopez, P. A., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y. P., Hilbrich, R., ... & Wießner, E. (2018, November). Microscopic traffic simulation using sumo. In 2018 21st international conference on intelligent transportation systems (ITSC) (pp. 2575-2582). IEEE.

[2] Mnih, Volodymyr, et al. "Human-level control through deep reinforcement learning." Nature 518.7540 (2015): 529- 533.

[3] Guo, M., Wang, P., Chan, C. Y., & Askary, S. (2019, October). A reinforcement learning approach for intelligent traffic signal control at urban intersections. In 2019 IEEE Intelligent Transportation Systems Conference (ITSC) (pp. 4242-4247). IEEE.

[4] S. El-Tantawy, B. Abdulhai, and H. Abdelgawad, “Design of reinforcement learning parameters for seamless application of adaptive traffic signal control,” J. Intell. Transp. Syst. Technol. Planning, Oper., vol. 18, no. 3, pp. 227–245, 2014.

[5] RoadwayVR 2025, GitHub – RoadwayVR/SUMO-Traffic-Simulator-Tutorial, GitHub, viewed 11August 2025, <https://github.com/RoadwayVR/SUMO-Traffic-Simulator-Tutorial>.

[6] Kovalchuk, G 2024, A Beginner’s Guide to Q-Learning: Understanding with a Simple Gridworld Example, Medium, viewed 11 August 2025, <https://medium.com/@goldengrisha/a-beginners-guide-to-q-learning-understanding-with-a-simple-gridworld-example-2b6736e7e2c9>.

[7] Amin, S 2024, Deep Q-Learning (DQN), Medium, viewed 11 August 2025, <https://medium.com/@samina.amin/deep-q-learning-dqn-71c109586bae>.